PYTHON 파일 입출력

PYTHON 파일 입출력

◆바이너리 & 텍스트

[데이터 분류]

데이터 타입	장점	단점
텍스트	- 텍스트 편집기로 편집 가능 - 데이터 설명 추가 가능	- 바이너리에 비해 크기가 큼 - 문자 인코딩 주의 (대부분 UTF-8)
바이너리	- 텍스트 데이터에 비해 크기 작음 - WEB에서 사용되는 데이터	- 텍스트 편집기로 편집 불가 - 데이터 설명 추가 불가

[텍스트 데이터 파일]

파일	특징
XML 파일	- 범용적인 형식, 웹 API 활용 형식
JSON 파일	- 자바스크립트 객체 표기 방법 기반 형식 파일 - 데이터 교환에 활용
YAML	- JSON 대용으로 사용, 어플리케이션 **설정 파일에** 많이 사용되는 파일
CSV/TSV	- WEB상에서 많이 사용되는 파일

PYTHON 파일 입출력

◆ 인코딩 & 디코딩

[사람 중심]

UNICODE

가을입니다.

인코딩(Encoding)

코드화/암호화

디코딩(Decoding)

역코드화/복호화

[기계 중심]

UTF-8, ASCII 등등 형식의 BYTE

\xea\xb0\x80\xec\x9d\x84\
xec\x9e\x85\xeb\x8b\x88\
xeb\x8b\xa4

0b100000000b11101100
0b100111010b10000100
0b111011000b10011110
0b100001010b11101011
0b100010110b10001000
0b111010110b10001011
0b101001000b10111000

PYTHON 파일 입출력

◆ 파일 읽기

```
file_Object = open( file , mode='r', encoding=None)
```

문자	의미
'r'	open for reading [DEFAULT]
'b'	binary mode
't'	text mode [DEFAULT]

PYTHON 파일 입출력

◆ 파일 읽기

- 읽을 데이터 양

전체 데이터 읽기	alldata = **f.read( )**
n바이트 데이터 읽기	alldata = **f.read(n)**

- 읽을 데이터 단위

줄 단위 1줄 데이터 읽기	alldata = **f.readline( )**
줄 단위 모든 데이터 읽기	alldata = **f.readlines( )**

PYTHON 파일 입출력

◆ 파일 쓰기

file_Object = **open(** file , mode='**w**', encoding=None**)**

문자	의미
'w'	open for writing, truncating the file first
'x'	open for exclusive creation, failing if the file already exists
'a'	open for writing, appending to the end of the file if it exists
'b'	binary mode
't'	text mode [DEFAULT]
'+'	open a disk file for updating (reading and writing)

PYTHON 파일 입출력

◆ 파일 읽기

- 쓰기 방식 => **자동 개행 안됨!!!**

전체 데이터 쓰기	charnums = **f.write( )**
줄단위 데이터 모두 쓰기	**f.writelines(list)**

PYTHON 파일 입출력

◆ 파일 포인터 위치

파일 위치 설정/이동	➔ f.seek(offset)	**# f.seek(0) 파일 처음**
파일 위치 읽기	➔ f.tell()	

파일 닫힘 여부 반환	➔ f.closed
파일모드 반환	➔ f.mode
파일이름 반환	➔ f.name

PYTHON 파일 입출력

◆ with 파일 as 구문

```
with  open(file , mode='w', encoding=None) as file_obj:
        file_obj.write( ' data ')
```

file_obj.close() 생략 가능

파일의 close() 자동으로 처리됨!!!